Edge AI Chess Engine : QAT, Distillation et Optimisation Bas Niveau sur Architecture ARM 32-bits

Ajyad HASSANI
Élève-Ingénieur (1A) – Télécom SudParis
Rôle : Algorithmique & Apprentissage Profond

Mars 2026

Table des matières

1 Résumé Exécutif **1**

2 Introduction **1**
2.1 Sélection de la plateforme : STM32F411 1
2.2 Capacité de calcul et architecture 1
2.3 Objectifs de l'EDD 1

3 Architecture du moteur d'évaluation (NNUE) **2**
3.1 Représentation de l'échiquier (One-Hot Encoding) 2
3.2 Formalisme de l'accumulateur 2
3.3 Incrémentalité 3
3.4 Gain de complexité et impact sur la mémoire Flash 3

4 Robustesse numérique et dimensionnement des registres **4**
4.1 Facteurs de quantification et seuils de saturation 4
4.2 Analyse de l'accumulateur 4
4.3 Analyse de la couche de sortie 5

5 Stratégie d'Entraînement sous Contraintes **6**
5.1 Objectif : apprendre sous contraintes matérielles et de capacité 6
5.2 Distillation : Stockfish comme oracle et définition des labels 6
5.3 Génération et sélection des données : qualité et diversité 7
5.4 QAT vs PTQ : justification et principe 7
5.5 Stabilisation : weight decay, clipping et descente projetée 8
5.6 Synthèse opérationnelle : pipeline unifié 9

6 Spécifications d'Intégration **9**
6.1 Algorithmique de recherche : Alpha-Bêta et formulation NegaMax 9
6.2 Génération de mouvements et optimisation bit-à-bit sur architecture 32 bits . . . 10
6.3 Accélération de l'inférence via les capacités DSP du Cortex-M4 11

1 Résumé Exécutif

Ce document détaille la conception d'un moteur d'échecs hybride sur **STM32F411**. L'innovation majeure réside dans l'intégration d'un réseau de neurones de type **NNUE** (*Efficiently Updatable Neural Network*) comme fonction d'évaluation, optimisé pour une exécution **sans recourir** au calcul en virgule flottante afin de maximiser le débit d'inférence sur STM32F411.

Afin de respecter la « loi d'airain » du matériel, le modèle suit une chaîne de transformation rigoureuse :

— **Architecture Frugale** : Une couche cachée limitée à 32 neurones pour minimiser l'empreinte mémoire.

— **Quantification Int16** : Une conversion intégrale en arithmétique entière 16 bits. Ce choix garantit une exécution déterministe et performante sur un microcontrôleur dépourvu d'accélérateurs dédiés au calcul matriciel.

— **Contraintes Matérielles** : Le système est dimensionné pour opérer dans une enveloppe stricte de **128 Ko de RAM** et **512 Ko de Flash**.

2 Introduction

2.1 Sélection de la plateforme : STM32F411

Le STM32F411 a été retenu pour son rapport performance/consommation optimal en contexte *Edge*. Bien que doté d'un cœur Cortex-M4, ses contraintes (fréquence, RAM/Flash, latence) imposent des choix algorithmiques orientés vers l'efficacité brute.

2.2 Capacité de calcul et architecture

Le système repose sur un processeur **ARM Cortex-M4 32 bits** monocœur cadencé à **100 MHz**.

— **Gestion du flux** : L'absence de multi-cœur impose un partage strict du temps processeur entre la recherche arborescente (Minimax) et l'inférence du réseau. L'optimisation du code est donc vitale au maintien de la profondeur de recherche.

— **Arithmétique entière** : Bien que le cœur Cortex-M4F dispose d'une FPU simple précision, l'inférence neuronale en *float* est délibérément évitée. Le choix **Int16** maximise le nombre de nœuds explorés par seconde, garantit un comportement déterministe et simplifie le dimensionnement numérique. Cette approche exploite la puissance native du jeu d'instructions 32 bits pour transformer l'inférence en une séquence d'opérations entières à faible latence et à haut débit.

2.3 Objectifs de l'EDD

Le présent document vise à démontrer que l'alliance d'un réseau de neurones et d'une quantification Int16 permet d'atteindre une précision d'évaluation supérieure aux heuristiques classiques, tout en respectant les limites thermiques et énergétiques d'un système embarqué.

3 Architecture du moteur d'évaluation (NNUE)

Le choix d'un réseau de neurones de type **NNUE** répond à la nécessité de dépasser les limites de précision des fonctions d'évaluation statiques traditionnelles tout en respectant les contraintes de latence d'un système embarqué. Cette architecture, composée d'une unique couche cachée couplée à une fonction d'activation de type **CReLu** (*Clipped ReLU*), effectue enfin un produit scalaire entre les activations et les poids de sortie afin de produire un score scalaire. L'ensemble repose sur la capacité de l'accumulateur à subir des mises à jour incrémentales induites par les différences entre deux positions consécutives.

3.1 Représentation de l'échiquier (One-Hot Encoding)

L'état physique de l'échiquier est projeté dans un espace vectoriel de dimension $N = 768$. Cet espace permet un mappage bijectif entre une configuration locale {pièce, couleur, case} et un index unique $k \in \{0, \ldots, 767\}$. L'indexation est régie par la fonction de hachage suivante :

$$k \;=\; (\,6 \cdot \text{couleur} \;+\; \text{pièce}\,) \cdot 64 \;+\; \text{case} \tag{1}$$

Les variables de cette fonction sont discrétisées selon les plages suivantes :

- **pièce** $\in \{0, 1, 2, 3, 4, 5\}$ correspondant respectivement à $\{P, N, B, R, Q, K\}$.
- **couleur** $\in \{0, 1\}$ où 0 représente les Blancs et 1 les Noirs.
- **case** $\in \{0, 1, \ldots, 63\}$ pour les 64 cases de l'échiquier.

Le vecteur d'entrée est défini comme un vecteur ligne binaire $\mathbf{x} \in \{0, 1\}^{1\times 768}$. Par construction, $\mathbf{x}$ est une représentation *One-Hot* : pour chaque case occupée, une seule composante parmi les 12 possibles est active (valeur 1), les autres étant nulles (0).

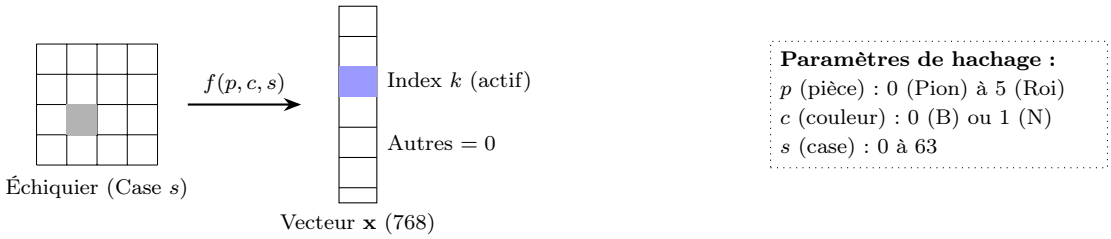


FIGURE 1 – Représentation One-Hot Encoding et projection vers l'index k.

3.2 Formalisme de l'accumulateur

L'unique couche cachée du réseau agit comme un accumulateur de caractéristiques. Pour garantir une implémentation logicielle directe sur architecture 32 bits, nous définissons l'accumulateur $\mathbf{A}$ comme un vecteur ligne de dimension 32.

Soit $\mathbf{W} \in \mathbb{Z}^{768\times 32}$ la matrice des poids synaptiques. Nous définissons $\mathbf{w}_k \in \mathbb{Z}^{1\times 32}$ comme le **vecteur ligne** correspondant à la k-ième ligne de la matrice $\mathbf{W}$. L'état de l'accumulateur est défini par :

$$\mathbf{A} = \mathbf{x}\mathbf{W} + \mathbf{b} = \left(\sum_{k, x_k = 1} \mathbf{w}_k \right) + \mathbf{b} \tag{2}$$

Où $\mathbf{b} \in \mathbb{Z}^{1\times 32}$ est le biais. L'accumulateur correspond mathématiquement à la somme vectorielle

des poids des caractéristiques actives.

3.3 Incrémentalité

La performance du moteur sur **STM32F411** repose sur l'exploitation de la corrélation entre deux positions successives. Lors d'un mouvement déplaçant une pièce d'une case source (s) vers une case destination (d), le vecteur $\mathbf{x}$ ne subit que deux modifications : l'indice i (pièce sortante) passe à 0 et l'indice j (pièce entrante) passe à 1.

Preuve par substitution : Soit $\mathbf{A}_t$ l'état de l'accumulateur à l'instant t. Par définition de la forme sommatoire :

$$\mathbf{A}_t = \left(\sum_{k \neq i, j, x_k = 1} \mathbf{w}_k \right) + \mathbf{w}_i + \mathbf{b} \tag{3}$$

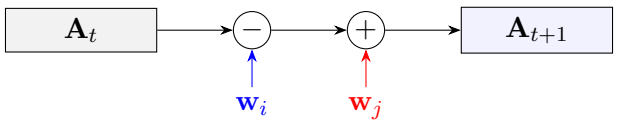
L'état suivant $\mathbf{A}_{t+1}$ s'écrit en désactivant l'indice i et en activant l'indice j :

$$\mathbf{A}_{t+1} = \left(\sum_{k \neq i, j, x_k = 1} \mathbf{w}_k \right) + \mathbf{w}_j + \mathbf{b} \tag{4}$$

En isolant le terme de l'état précédent dans l'équation, nous identifions directement la relation de récurrence :

$$\mathbf{A}_{t+1} = \underbrace{\left(\sum_{k \neq i, j, x_k = 1} \mathbf{w}_k + \mathbf{w}_i + \mathbf{b} \right)}_{\mathbf{A}_t} - \mathbf{w}_i + \mathbf{w}_j \tag{5}$$

D'où la relation fondamentale : $\mathbf{A}_{t+1} = \mathbf{A}_t - \mathbf{w}_i + \mathbf{w}_j$.



Cas simple (sans capture / roque / promotion) :
$2 \times HL$ opérations Int16
$\approx$ 64 (32 subs + 32 adds)
Capture / roque / promotion : facteur constant supplémentaire,
mais le coût reste en $O(HL)$

FIGURE 2 – Mécanisme de mise à jour différentielle de l'accumulateur.

3.4 Gain de complexité et impact sur la mémoire Flash

Cette démonstration prouve que la mise à jour incrémentale est en $O(HL)$ et, avec $HL = 32$, le coût est constant. Dans le cas simple (sans capture/roque/promotion), le coût CPU par nœud est d'environ $2HL \approx 64$ opérations entières. Sur le plan matériel, ce choix est validé par son empreinte sur la Flash (512 Ko) :

— **Stockage de la matrice W** : $768 \times 32 \times 2$ octets (Int16) = 49 152 octets.
— **Occupation relative** : L'accumulateur n'occupe que **9,6 % de la Flash totale**.

4 Robustesse numérique et dimensionnement des registres

L'implémentation de l'inférence sur l'architecture ARM Cortex-M4 repose sur une arithmétique entière stricte. Afin de prévenir tout risque de dépassement de capacité (*overflow*) au sein des registres 16 et 32 bits, des contraintes sur les plages de valeurs ont été définies en amont de la quantification. Cette section détaille les intervalles imposés à chaque strate du réseau et démontre mathématiquement la stabilité du graphe de calcul.

4.1 Facteurs de quantification et seuils de saturation

Le passage des paramètres de l'espace continu (nombres à virgule flottante) vers l'espace discret (entiers) s'articule autour de deux constantes de mise à l'échelle : $Q_A = 127$ et $Q_B = 64$.

Afin de borner la magnitude des entiers générés, des coefficients de clippage (notés C) sont appliqués aux paramètres réels (poids et biais) préalablement à leur multiplication par les facteurs de quantification. La transformation s'opère de manière distincte selon la couche du réseau.

A. Couche cachée (Accumulateur) Pour la première strate du réseau, un coefficient de clippage $C_1 = 7$ est imposé. Les valeurs réelles des poids ($W_{1,\text{réel}}$) et des biais ($B_{1,\text{réel}}$) sont ainsi contraintes dans l'intervalle $[-7, 7]$. La conversion en entiers utilise le facteur Q_A :

- **Poids de l'accumulateur :** $W_{1,int} = \text{round}(W_{1,\text{réel}} \cdot Q_A)$. L'intervalle entier résultant est $[-7 \times 127, +7 \times 127]$, soit $[-889, 889]$.
- **Biais de l'accumulateur :** La même règle s'applique, $B_{1,int} = \text{round}(B_{1,\text{réel}} \cdot Q_A)$. L'intervalle est strictement identique : $[-889, 889]$.

B. Couche de sortie Pour la seconde strate, chargée d'agréger les données de l'accumulateur, un coefficient de clippage plus restrictif $C_2 = 2$ est appliqué. Les opérations de quantification diffèrent entre les poids et le biais afin de maintenir la cohérence dimensionnelle lors du produit scalaire :

- **Poids de sortie :** Les valeurs réelles ($W_{2,\text{réel}} \in [-2, 2]$) sont quantifiées via le facteur Q_B. La relation $W_{2,int} = \text{round}(W_{2,\text{réel}} \cdot Q_B)$ établit un intervalle entier de $[-2 \times 64, +2 \times 64]$, soit $[-128, 128]$.
- **Biais de sortie :** Pour que le biais puisse être additionné au résultat du produit scalaire (lequel implique des activations à l'échelle Q_A et des poids à l'échelle Q_B), il doit être quantifié selon le produit des deux facteurs. Sous la contrainte du seuil C_2, la relation s'écrit $B_{\text{out},int} = \text{round}(B_{\text{out,réel}} \cdot Q_A \cdot Q_B)$. L'intervalle entier résultant est $[-2 \times (127 \times 64), +2 \times (127 \times 64)]$, soit $[-16\,256, 16\,256]$.

4.2 Analyse de l'accumulateur

L'état de l'accumulateur est stocké dans un tableau d'entiers 16 bits signés (`int16_t`), dont la plage de valeurs s'étend de $-32\,768$ à $+32\,767$.

La valeur d'une composante de l'accumulateur dépend du biais B_1 et de la somme des poids W_1 correspondant aux caractéristiques actives. Aux échecs, le nombre maximum de pièces simultanément présentes sur l'échiquier est $N_{max} = 32$. La valeur maximale absolue théorique est

donnée par l'équation :

$$|A|_{max} = |B_1| + \sum_{k=1}^{32} |W_{1,k}| \tag{6}$$

En substituant les variables par les bornes supérieures définies à la section précédente :

$$|A|_{max} \leq 889 + (32 \times 889) = 33 \times 889 = \mathbf{29\,337} \tag{7}$$

La condition $29\,337 \leq 32\,767$ est vérifiée. Cette limite garantit que la mise à jour incrémentale de l'accumulateur peut être exécutée en continu sans générer de dépassement de capacité et sans nécessiter l'intégration d'instructions de vérification conditionnelle.

4.3 Analyse de la couche de sortie

L'évaluation combine deux accumulateurs de taille $HL = 32$ (une perspective "Blanc" et une perspective "Noir"), concaténés en un vecteur de taille $2HL = 64$ avant le produit scalaire final. Le calcul du score de sortie s'effectue au sein d'un registre 32 bits signé (`int32_t`). Les activations a_i transmises par la couche cachée sont préalablement bornées par la fonction **CReLu** dans l'intervalle $[0, Q_A]$, soit $[0, 127]$.

Calcul du score brut Le score intermédiaire S agrège les contributions des 64 neurones (32 issus de la perspective blanche, 32 de la perspective noire) et y ajoute le biais de sortie B_{out}. Sa valeur maximale absolue s'établit à :

$$|S|_{max} = |B_{\text{out}}| + \sum_{i=1}^{64} (a_i \cdot |W_{2,i}|) \tag{8}$$

$$|S|_{max} = 16\,256 + (64 \times 127 \times 128) = 16\,256 + 1\,040\,384 = \mathbf{1\,056\,640} \tag{9}$$

Mise à l'échelle finale Avant l'étape de déquantification, le score intermédiaire S est multiplié par la constante SCALE = 2000. Cette opération définit la limite dynamique maximale du système :

$$V_{\text{scaled_max}} = |S|_{max} \times 2000 = 1\,056\,640 \times 2000 = \mathbf{2\,113\,280\,000} \tag{10}$$

La valeur maximale représentable par un registre `int32_t` étant de 2 147 483 647, le système exploite, dans la configuration la plus extrême théoriquement possible, 98,4 % de la plage de valeurs disponible.

L'évaluation finale est ensuite obtenue par la division de cette valeur par le produit $Q_A \times Q_B$:

$$\text{Evaluation} = \frac{V_{\text{scaled}}}{Q_A \times Q_B} \tag{11}$$

Cette démonstration établit que les intervalles de clippage $C_1 = 7$ et $C_2 = 2$ bornent mathématiquement le réseau en dessous des limites matérielles du STM32F411, ce qui garantit, sous les hypothèses de bornage, l'absence de dépassement de capacité dans le chemin de calcul.

5 Stratégie d'Entraînement sous Contraintes

5.1 Objectif : apprendre sous contraintes matérielles et de capacité

La section précédente a dimensionné le graphe de calcul de manière à garantir une exécution robuste sur **STM32F411** : choix des facteurs de quantification ($Q_A = 127$, $Q_B = 64$), bornage des paramètres, et démonstration de l'absence d'*overflow* dans les registres `int16_t` et `int32_t`. Ce résultat n'est valide que si l'entraînement respecte effectivement les hypothèses de bornage utilisées dans les démonstrations.

Deux contraintes majeures imposent donc une stratégie d'apprentissage spécifique :

— **Contrainte matérielle** : le modèle final est exécuté en arithmétique entière. Toute dérive des poids/biais au-delà des plages prévues réintroduit un risque d'instabilité numérique (saturation, perte d'information, voire dépassement de capacité).

— **Contrainte de capacité** : avec une unique couche cachée de **32 neurones**, le réseau ne dispose pas de redondance interne suffisante pour absorber les erreurs induites par la quantification ou par des labels bruités. La qualité des données et la stabilité de l'optimisation sont donc des facteurs déterminants.

L'objectif de cette section est de formaliser la chaîne d'entraînement qui permet d'obtenir un NNUE : (i) compatible avec l'exécution entière sur microcontrôleur, (ii) stable numériquement, et (iii) suffisamment précis malgré sa faible capacité.

5.2 Distillation : Stockfish comme oracle et définition des labels

Le réseau est entraîné par **distillation** à partir d'un moteur expert (Stockfish), utilisé comme oracle. À chaque position d'entraînement, Stockfish fournit une évaluation en **centipawns** (cp), utilisée comme cible supervisée.

Budget de calcul (nodes = 5000). L'oracle est configuré avec un budget fixe en nombre de nœuds explorés (noté `nodes`). Fixer `nodes=5000` permet d'obtenir des labels plus stables qu'une évaluation superficielle, tout en conservant un débit de génération compatible avec la production d'un dataset volumineux.

Deux meilleurs coups : mesure du caractère forçant. Pour chaque position, l'oracle calcule les **deux meilleurs coups** (maximisant l'évaluation de l'oracle, sous budget `nodes=5000`) et associe à chacun un score en **centipawns** :

— s_1 : score (en cp) du meilleur coup,

— s_2 : score (en cp) du second meilleur coup.

On définit alors une mesure de *sharpness* :

$$\Delta = |s_1 - s_2|. \tag{12}$$

Une faible Δ caractérise une position où plusieurs coups se valent, tandis qu'une forte Δ indique qu'un coup se détache nettement des alternatives, typiquement dans des situations plus tactiques ou contraintes.

Bornage et normalisation des cibles. Afin de stabiliser l'optimisation et de rendre la cible compatible avec la dynamique du réseau, le score est borné puis normalisé :

$$y = \frac{\mathrm{clip}(s_1, -C, C)}{C}, \qquad C = 2000. \tag{13}$$

Cette normalisation borne typiquement y dans l'intervalle $[-1, 1]$ et rend la fonction de perte (de type Huber/SmoothL1) numériquement stable.

5.3 Génération et sélection des données : qualité et diversité

Avec une unique couche cachée de 32 neurones, la qualité des données est un facteur déterminant : le modèle ne peut pas compenser un dataset fortement bruité ou déséquilibré. La génération suit donc un protocole orienté « **qualité et diversité** ».

Génération de trajectoires réalistes. Les positions sont obtenues en générant des parties synthétiques. La politique de jeu cherche à rester plausible tout en évitant la répétitivité :

— un début de partie partiellement randomisé permet de diversifier les structures,
— puis les coups sont choisis dans un ensemble restreint de coups de bonne qualité proposés par le moteur (afin d'éviter des positions peu représentatives des distributions de parties réelles).

Rééquilibrage de la distribution des scores. Les positions « naturelles » sont majoritairement proches de 0 cp. Sans correction, le dataset serait dominé par des exemples quasi neutres et le modèle apprendrait essentiellement à prédire 0. Un rééquilibrage est donc appliqué : les positions faiblement avantageuses (petit $|s_1|$) sont sous-échantillonnées, tandis que les positions où $|s_1|$ est plus marqué sont surreprésentées.

Contrôle du niveau tactique via Δ. Un excès de positions extrêmement forçantes (très grande Δ) peut introduire une variabilité difficile à modéliser pour un réseau minimaliste. À l'inverse, un dataset trop « plat » produit une évaluation peu discriminante. La mesure Δ est donc utilisée pour contrôler la part de positions tactiques : elles ne sont pas supprimées, mais intégrées de manière contrôlée (par filtrage probabiliste ou quotas par tranches de Δ).

Sous-échantillonnage du tout début de partie. Les toutes premières positions sont très similaires d'une partie à l'autre et apportent peu d'information utile au modèle. Elles sont donc sous-échantillonnées au profit de positions plus informatives (milieux de partie, structures de pions variées, transitions).

5.4 QAT vs PTQ : justification et principe

La quantification finale repose sur le schéma défini dans la section précédente ($Q_A = 127$, $Q_B = 64$), et sur une inférence entière stricte. Deux approches peuvent conduire à un modèle quantifié :

— **PTQ** : entraînement en flottant, quantification uniquement en fin de chaîne,
— **QAT** : entraînement en simulant la quantification pendant l'apprentissage.

Dans le cadre d'un réseau très frugal (**HL=32**), la PTQ conduit fréquemment à une perte de performance, car l'arrondi et la saturation modifient sensiblement la fonction d'évaluation. Le **QAT** est donc retenu afin d'aligner l'apprentissage sur le comportement réel du modèle déployé.

Fake quantification et approximation de gradient. Le QAT consiste à introduire, dans le *forward* d'entraînement, une **fausse quantification** (*fake quant*) qui reproduit les opérations de mise à l'échelle, d'arrondi et de saturation. Le *backward* utilise une approximation de gradient (de type *Straight-Through Estimator*, STE) afin de permettre la descente de gradient malgré la non-dérivabilité de l'arrondi.

5.5 Stabilisation : weight decay, clipping et descente projetée

Afin de garantir la compatibilité du modèle avec les bornes numériques démontrées en section précédente, l'entraînement intègre des mécanismes explicites de stabilisation.

Weight decay. Une régularisation de type L_2 (implémentée via AdamW) limite la croissance de la norme des paramètres. Cela évite que le modèle tente de compenser sa faible capacité par une explosion des poids, comportement particulièrement destructeur après quantification (saturation et perte d'information).

Gradient clipping. Le clipping des gradients borne les mises à jour trop violentes, notamment sur des batchs difficiles (positions tactiques). Il stabilise la convergence et réduit les oscillations.

Descente de gradient projetée (projection des paramètres). Après chaque étape d'optimisation, les paramètres sont projetés dans un hyper-rectangle de contraintes cohérent avec le dimensionnement numérique :

— $W_1, B_1 \in [-7, 7]$ (couche d'accumulateur),
— $W_2, B_2 \in [-2, 2]$ (couche de sortie).

Mathématiquement :

$$\theta \leftarrow \Pi_{\mathcal{C}} \left(\theta - \eta \nabla L(\theta) \right), \tag{14}$$

où $\Pi_{\mathcal{C}}$ est la projection sur l'ensemble admissible $\mathcal{C}$. Cette étape assure que les poids restent compatibles avec le schéma de quantification et que les intervalles finaux (en entier) respectent les bornes nécessaires à l'absence d'*overflow*.

```python
def secure_weights(model):
    # Bornes en float (avant quantification)
    ACC_W_MIN, ACC_W_MAX = -7.0, 7.0
    ACC_B_MIN, ACC_B_MAX = -7.0, 7.0

    OUT_W_MIN, OUT_W_MAX = -2.0, 2.0
    OUT_B_MIN, OUT_B_MAX = -2.0, 2.0

    with torch.no_grad():
        model.acc_weight.weight.data.clamp_(ACC_W_MIN, ACC_W_MAX)
        if model.acc_weight.bias is not None:
            model.acc_weight.bias.data.clamp_(ACC_B_MIN, ACC_B_MAX)

        model.output_weight.weight.data.clamp_(OUT_W_MIN, OUT_W_MAX)
        if model.output_weight.bias is not None:
```

```
            model.output_weight.bias.data.clamp_(OUT_B_MIN, OUT_B_MAX)
```

Listing 1 – Projection des paramètres (`secure_weights`) : descente de gradient projetée.

5.6 Synthèse opérationnelle : pipeline unifié

La chaîne d'entraînement sous contraintes est structurée comme suit :

1. **Génération** de positions diversifiées et plausibles (trajectoires synthétiques) et **sous-échantillonnage** des positions redondantes (début de partie).
2. **Labellisation** par distillation Stockfish avec budget fixe `nodes=5000` : pour chaque position, calcul des **deux meilleurs coups** et de leurs scores s_1 et s_2 (en cp), puis calcul de $\Delta = |s_1 - s_2|$.
3. **Filtrage et échantillonnage** : bornage des scores via C, normalisation $y = s_1/C$, rééquilibrage de la distribution de $|y|$, et contrôle du niveau tactique via Δ.
4. **Entraînement principal (boucle unique)** : **QAT** (fake quant + STE) *conjointement* à la stabilisation (*weight decay*, *gradient clipping*) et à la **projection** des paramètres (`secure_weights`) après mise à jour.
5. **Export** des poids quantifiés cohérents avec Q_A et Q_B (et biais à l'échelle $Q_A Q_B$), puis validation de cohérence float→entier sur un ensemble de positions de test.

6 Spécifications d'Intégration

Cette section constitue le cadre technique à destination de l'équipe de développement C/C++. Elle précise les choix d'intégration nécessaires pour que le moteur d'échecs exploite correctement les optimisations définies dans les sections précédentes, tout en restant compatible avec les contraintes de calcul du STM32F411. L'objectif est ici de traduire les choix théoriques en exigences d'implémentation cohérentes avec une exécution embarquée, frugale et déterministe.

6.1 Algorithmique de recherche : Alpha-Bêta et formulation NegaMax

L'algorithme de recherche recommandé pour la première implémentation est une recherche **Alpha-Bêta**, formulée sous la forme **NegaMax**. Ce choix est motivé par son excellent compromis entre simplicité d'implémentation, coût de calcul et profondeur de recherche atteignable sur une cible embarquée. À profondeur égale, Alpha-Bêta permet de réduire fortement le nombre de nœuds explorés par rapport à une recherche exhaustive naïve, tout en conservant le même résultat qu'un schéma Minimax classique hors heuristiques sélectives.

La formulation **NegaMax** est retenue car elle s'aligne naturellement avec le comportement du réseau **NNUE**, qui renvoie une évaluation exprimée selon une perspective donnée. Dans ce cadre, il devient possible d'unifier la recherche autour d'une seule fonction récursive, ce qui simplifie l'implémentation et réduit la complexité logique du moteur. Concrètement, chaque nœud est évalué en maximisant l'opposé de la valeur retournée par les nœuds fils, ce qui permet de conserver une structure de code compacte et cohérente avec l'évaluation neuronale.

La performance pratique d'une recherche Alpha-Bêta dépend cependant fortement de la qualité de l'ordonnancement des coups et des mécanismes d'élagage associés. L'implémentation devra donc intégrer prioritairement les optimisations suivantes :

- **Iterative Deepening**, afin d'obtenir à tout instant une meilleure réponse disponible à profondeur croissante. Cette stratégie permet également d'améliorer progressivement l'ordonnancement des coups au fil des itérations.
- **Move Ordering**, pour tester en priorité les coups les plus prometteurs et provoquer les coupures le plus tôt possible. Sur une recherche Alpha-Bêta, la qualité de cet ordonnancement conditionne directement l'efficacité réelle de l'élagage.
- **Killer Heuristic** et **History Heuristic**, afin d'exploiter les régularités observées dans l'arbre de recherche. Ces heuristiques permettent de prioriser certains coups ayant déjà montré leur efficacité dans des contextes similaires.

Dans cette architecture, le **NNUE** constitue la fonction d'évaluation statique de référence. Il doit être intégré de façon cohérente avec la recherche, de manière à guider le moteur sans introduire de divergence entre la logique de recherche et la logique d'évaluation. L'ensemble recherche + évaluation doit donc être conçu comme un système unifié : Alpha-Bêta réduit l'espace exploré, tandis que le NNUE fournit une estimation rapide et stable de la qualité des positions atteintes.

6.2 Génération de mouvements et optimisation bit-à-bit sur architecture 32 bits

La représentation de l'échiquier repose sur des **bitboards**, c'est-à-dire des entiers de 64 bits où chaque bit représente une case. Ce choix reste pertinent sur STM32F411, car il permet d'exprimer de manière compacte et efficace l'occupation des pièces, les masques d'attaque et de nombreuses opérations échiquéennes.

Néanmoins, le STM32F411 étant une architecture **32 bits**, la manipulation de données 64 bits induit un surcoût par rapport à une cible 64 bits native. L'implémentation du générateur de coups devra donc être pensée pour limiter autant que possible ce surcoût, tout en conservant les avantages algorithmiques des bitboards.

En pratique, cela implique :

- un usage intensif des opérations logiques bit-à-bit ;
- une structuration compacte des masques et des calculs d'attaque ;
- et, lorsque cela est pertinent, l'exploitation de techniques d'optimisation de type **SWAR** adaptées au cœur Cortex-M4.

L'objectif n'est pas d'ajouter une complexité inutile, mais de garantir que la génération de coups reste suffisamment efficace pour ne pas annuler les gains obtenus par la frugalité du NNUE. Cette exigence est essentielle, car la génération de coups est exécutée à très haute fréquence dans la boucle de recherche ; toute inefficacité à ce niveau réduirait directement la profondeur atteignable par le moteur.

6.3 Accélération de l'inférence via les capacités DSP du Cortex-M4

L'architecture du NNUE a été définie de manière à être compatible avec une exécution entière efficace sur **Cortex-M4**. Le choix d'une quantification stricte en **Int16** ne répond pas uniquement à une contrainte de mémoire : il vise également à rendre possible une exploitation efficace des capacités de calcul entier du microcontrôleur.

En conséquence, les parties critiques de l'inférence — en particulier :

— la **mise à jour incrémentale de l'accumulateur** ;

— et le **calcul final de la couche de sortie** —

doivent être implémentées de manière à tirer parti des mécanismes de calcul optimisés disponibles sur le cœur ARM, plutôt que de reposer uniquement sur une version scalaire naïve.

L'exigence n'est pas ici d'imposer une implémentation bas niveau spécifique dans le détail, mais de fixer un principe clair : les boucles critiques du NNUE doivent être structurées pour bénéficier des capacités DSP du Cortex-M4, de façon à réduire le coût du produit scalaire et des opérations d'accumulation. Cela permet de préserver, au moment de l'exécution réelle, le bénéfice attendu du choix NNUE + quantification entière.

Autrement dit, le modèle ne doit pas seulement être mathématiquement compatible avec le matériel ; son intégration logicielle doit également être pensée pour exploiter efficacement les ressources de calcul disponibles. Sans cette cohérence entre architecture réseau et implémentation embarquée, une part importante du gain de performance attendu serait perdue.